

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

INDIVIDUAL PROJECT ON

STUDENT ACADEMIC & CONTINUOUS EVALUATION SYSTEM

P R O J E C T R E P O R T

Submitted by

SUSHMITHA T
25ITR150

JAVA ASSIGNMENT

AIM:

Development of Kongu Engineering College (R2024) Student Academic & Continuous Evaluation System using Java JDBC.

OBJECTIVE:

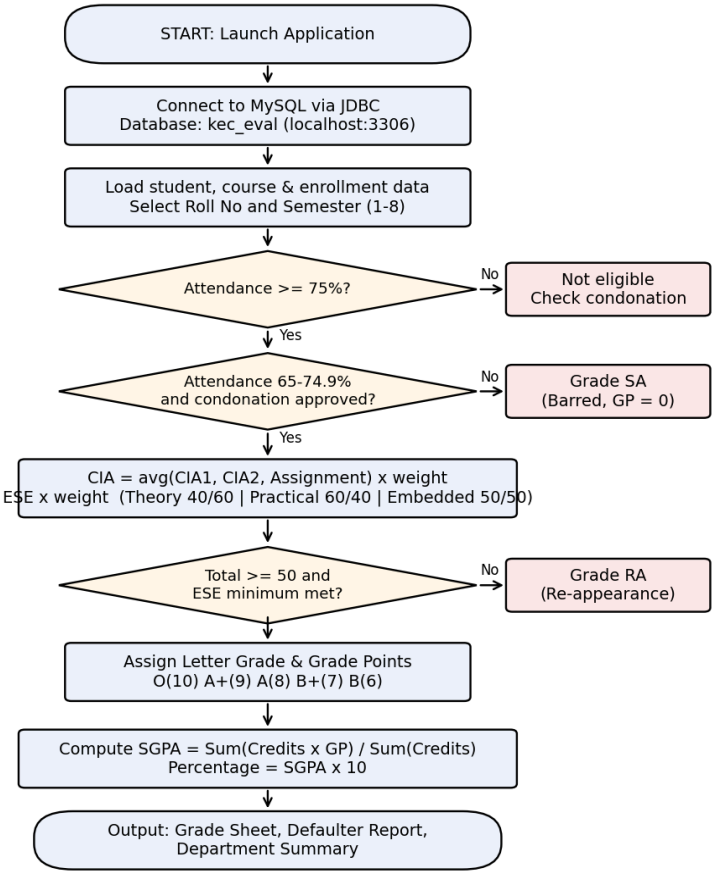
To design and develop a database-driven Java application using JDBC to manage student records, perform incremental marks entry (Continuous Internal Assessments and End Semester Examinations), enforce attendance eligibility rules, and generate formal academic performance reports according to Kongu Engineering College (Autonomous) - Regulation 2024.

SYSTEM REQUIREMENTS & SPECIFICATIONS:

- Student Registration: roll number, name, department and current semester.
- Course Catalog: course code, title, credits and course type (THEORY, PRACTICAL, EMBEDDED).
- Enrollment & Attendance: classes held versus classes attended for each course and semester.
- Incremental Marks Entry: Phase 1 (CIA Test 1 and 2), Phase 2 (Assignment/Quiz/Lab), Phase 3 (ESE marks or Absent).

SGPA = Sum(Course Credits x Grade Points) / Sum(Total Enrolled Credits). Equivalent Percentage = SGPA x 10. The CIA component (out of 100) is the average of CIA-1, CIA-2 and Assignment marks, which is then scaled by the course-type weightage.

FLOW CHART:


START: Launch Application
Connect to MySQL via JDBC
Database: kec_eval (localhost:3306)
Load student, course & enrollment data
Select Roll No and Semester (1-8)
Attendance >= 75%?
No
Not eligible
Check condonation
Yes
Attendance 65-74.9%
and condonation approved?
No
Grade SA
(Barred, GP = 0)
Yes
CIA = avg(CIA1, CIA2, Assignment) x weight
ESE x weight (Theory 40/60 | Practical 60/40 | Embedded 50/50)
Total >= 50 and
ESE minimum met?
No
Grade RA
(Re-appearance)
Assign Letter Grade & Grade Points
O(10) A+(9) A(8) B+(7) B(6)
Compute SGPA = Sum(Credits x GP) / Sum(Credits)
Percentage = SGPA x 10
Output: Grade Sheet, Defaulter Report,
Department Summary

PROCEDURE:

- Load the MySQL Connector/J driver so the Java application can communicate with the MySQL database.
- Establish a connection to the kec_eval database using DriverManager.getConnection().
- Store and retrieve student details (roll number, name, department, semester) and course details (code, title, credits, type).
- Enroll students in semester courses, record classes held and attended, and calculate the attendance percentage.
- Verify examination eligibility: 75% or above is eligible, 65%-74.9% requires medical condonation, below 65% is Shortage of Attendance (SA).
- Record CIA-1, CIA-2 and Assignment marks in phases and calculate the scaled internal marks using the course-type weightage.
- Record End Semester Examination marks out of 100 (or mark absence), verify the ESE minimum, and scale the marks.
- Calculate the total, assign the letter grade and grade points, and mark RA if the student fails or scores below 50.
- Compute SGPA and equivalent percentage, and generate the grade sheet, attendance defaulter report and department-wise result summary on the console.

DATABASE SCHEMA:

1. STUDENTS TABLE

```
mysql> DESC students;
+------------------+--------------+------+-----+-------------------+-------------------+
| Field            | Type         | Null | Key | Default           | Extra             |
+------------------+--------------+------+-----+-------------------+-------------------+
| roll_no          | varchar(20)  | NO   | PRI | NULL              |                   |
| name             | varchar(100) | NO   |     | NULL              |                   |
| department       | varchar(20)  | NO   |     | NULL              |                   |
| current_semester | int          | NO   |     | 3                 |                   |
| created_at       | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+--------------+------+-----+-------------------+-------------------+
5 rows in set (0.02 sec)
```

2. COURSES TABLE

```
mysql> DESC courses;
+-------------+-----------------------------------------+------+-----+-------------------+-------------------+
| Field       | Type                                    | Null | Key | Default           | Extra             |
+-------------+-----------------------------------------+------+-----+-------------------+-------------------+
| course_code | varchar(20)                             | NO   | PRI | NULL              |                   |
| title       | varchar(150)                            | NO   |     | NULL              |                   |
| credits     | int                                     | NO   |     | NULL              |                   |
| course_type | enum('THEORY','PRACTICAL','EMBEDDED')   | NO   |     | NULL              |                   |
| semester    | int                                     | NO   |     | 3                 |                   |
| created_at  | timestamp                               | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+-----------------------------------------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)
```

3. ENROLLMENTS TABLE

```
mysql> DESC enrollments;
+------------------+-------------+------+-----+-------------------+-------------------+
| Field            | Type        | Null | Key | Default           | Extra             |
+------------------+-------------+------+-----+-------------------+-------------------+
| enrollment_id    | int         | NO   | PRI | NULL              | auto_increment    |
| roll_no          | varchar(20) | NO   | MUL | NULL              |                   |
| course_code      | varchar(20) | NO   | MUL | NULL              |                   |
| semester         | int         | NO   |     | 3                 |                   |
| total_classes    | int         | NO   |     | 0                 |                   |
| attended_classes | int         | NO   |     | 0                 |                   |
| is_condoned      | tinyint(1)  | NO   |     | 0                 |                   |
| created_at       | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+-------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)
```

4. ASSESSMENT_MARKS TABLE

```
mysql> DESC assessment_marks;
+-----------------+--------------+------+-----+-------------------+-----------------------------------------------+
| Field           | Type         | Null | Key | Default           | Extra                                         |
+-----------------+--------------+------+-----+-------------------+-----------------------------------------------+
| mark_id         | int          | NO   | PRI | NULL              | auto_increment                                |
| enrollment_id   | int          | NO   | UNI | NULL              |                                               |
| cia1_marks      | decimal(5,2) | YES  |     | NULL              |                                               |
| cia2_marks      | decimal(5,2) | YES  |     | NULL              |                                               |
| assignment_marks| decimal(5,2) | YES  |     | NULL              |                                               |
| ese_marks       | decimal(5,2) | YES  |     | NULL              |                                               |
| is_ese_absent   | tinyint(1)   | NO   |     | 0                 |                                               |
| updated_at      | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+-----------------+--------------+------+-----+-------------------+-----------------------------------------------+
8 rows in set (0.00 sec)
```

SOURCE CODE:

DBCONNECTION.JAVA:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/** JDBC connection management. Override defaults with env vars
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD. */
public class DBConnection {
    private static String env(String key, String def) {
        String v = System.getenv(key);
        return v != null ? v : def;
    }

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.err.println("[ERROR] MySQL Connector/J not on
classpath: " + e.getMessage());
        }
    }

    public static Connection getConnection() throws SQLException {
        String url = "jdbc:mysql://" + env("DB_HOST", "localhost")
+ ":" + env("DB_PORT", "3306") + "/"
                + env("DB_NAME", "kec_eval") +
"?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";
        return DriverManager.getConnection(url, env("DB_USER",
"root"), env("DB_PASSWORD", "root"));
    }

    public static boolean testConnection() {
        try (Connection c = getConnection()) {
            return c != null && !c.isClosed();
        } catch (SQLException e) {
            System.err.println("[ERROR] " + e.getMessage());
            return false;
        }
    }
}
```

ACADEMICDAO.JAVA:

```
import java.sql.*;
import java.util.*;

/** SQL CRUD + KEC R2024 evaluation engine. */
public class AcademicDAO {

    /** One evaluated course result for a student. */
    public static class Result {
        String roll, name, dept, code, title, type;
        int credits;
        double attPct;
        Double cia1, cia2, asg, ese;
        boolean absent, condoned;
        double scaledCia, scaledEse, total;
        String grade = "-", status = "PENDING";
        int gp;
        boolean pending;
    }

    // ---------- Data entry ----------
    public boolean addStudent(String roll, String name, String
dept, int sem) {
        String sql = "INSERT INTO
students(roll_no,name,department,current_semester)
VALUES(?,?,?,?)";
        try (Connection c = DBConnection.getConnection();
PreparedStatement p = c.prepareStatement(sql)) {
            p.setString(1, roll); p.setString(2, name);
p.setString(3, dept.toUpperCase()); p.setInt(4, sem);
            return p.executeUpdate() > 0;
        } catch (SQLException e) { System.out.println("[ERROR] " +
e.getMessage()); return false; }
    }

    public boolean addCourse(String code, String title, int
credits, String type, int sem) {
        String sql = "INSERT INTO
courses(course_code,title,credits,course_type,semester)
VALUES(?,?,?,?,?)";
        try (Connection c = DBConnection.getConnection();
PreparedStatement p = c.prepareStatement(sql)) {
            p.setString(1, code); p.setString(2, title);
p.setInt(3, credits); p.setString(4, type); p.setInt(5, sem);
            return p.executeUpdate() > 0;
        } catch (SQLException e) { System.out.println("[ERROR] " +
e.getMessage()); return false; }
    }

    /** Enrolls (or updates attendance of) a student; returns
enrollment id or -1. */
    public int enroll(String roll, String course, int sem, int
total, int attended) {
        String up = "INSERT INTO
enrollments(roll_no,course_code,semester,total_classes,attended_cl
asses) VALUES(?,?,?,?,?) "
                + "ON DUPLICATE KEY UPDATE
total_classes=VALUES(total_classes),
attended_classes=VALUES(attended_classes)";
```

```
        try (Connection c = DBConnection.getConnection();
PreparedStatement p = c.prepareStatement(up)) {
            p.setString(1, roll); p.setString(2, course);
p.setInt(3, sem); p.setInt(4, total); p.setInt(5, attended);
            p.executeUpdate();
            int id = getEnrollmentId(roll, course, sem);
            try (PreparedStatement m = c.prepareStatement("INSERT
IGNORE INTO assessment_marks(enrollment_id) VALUES(?)")) {
                m.setInt(1, id); m.executeUpdate();
            }
            return id;
        } catch (SQLException e) { System.out.println("[ERROR] " +
e.getMessage()); return -1; }
    }

    public int getEnrollmentId(String roll, String course, int
sem) {
        String sql = "SELECT enrollment_id FROM enrollments WHERE
roll_no=? AND course_code=? AND semester=?";
        try (Connection c = DBConnection.getConnection();
PreparedStatement p = c.prepareStatement(sql)) {
            p.setString(1, roll); p.setString(2, course);
p.setInt(3, sem);
            try (ResultSet r = p.executeQuery()) { return r.next()
? r.getInt(1) : -1; }
        } catch (SQLException e) { System.out.println("[ERROR] " +
e.getMessage()); return -1; }
    }

    public boolean setCondonation(int enrollmentId, boolean
approved) {
        return update("UPDATE enrollments SET is_condoned=? WHERE
enrollment_id=?", approved ? 1 : 0, enrollmentId);
    }

    // ---------- Incremental marks ----------
    public boolean recordPhase1Marks(int id, double cia1, double
cia2) {
        if (!valid(cia1) || !valid(cia2)) return false;
        return update("UPDATE assessment_marks SET cia1_marks=?,
cia2_marks=? WHERE enrollment_id=?", cia1, cia2, id);
    }

    public boolean recordPhase2Marks(int id, double asg) {
        if (!valid(asg)) return false;
        return update("UPDATE assessment_marks SET
assignment_marks=? WHERE enrollment_id=?", asg, id);
    }

    public boolean recordPhase3Marks(int id, Double ese, boolean
absent) {
        if (absent) return update("UPDATE assessment_marks SET
ese_marks=NULL, is_ese_absent=1 WHERE enrollment_id=?", id);
        if (ese == null || !valid(ese)) return false;
        return update("UPDATE assessment_marks SET ese_marks=?,
is_ese_absent=0 WHERE enrollment_id=?", ese, id);
    }

    private boolean valid(double m) {
```

```
        if (m < 0 || m > 100) { System.out.println("[ERROR] Marks
must be between 0 and 100."); return false; }
        return true;
    }

    private boolean update(String sql, Object... args) {
        try (Connection c = DBConnection.getConnection();
PreparedStatement p = c.prepareStatement(sql)) {
            for (int i = 0; i < args.length; i++) p.setObject(i +
1, args[i]);
            return p.executeUpdate() > 0;
        } catch (SQLException e) { System.out.println("[ERROR] " +
e.getMessage()); return false; }
    }

    // ---------- Evaluation engine (KEC R2024)
----------
    /** Applies attendance rules, weightage, ESE threshold and
grading. Expects r.attPct to be set. */
    public static void evaluate(Result r) {
        // Attendance filter: <65 -> SA; 65-74.9 needs medical
condonation; >=75 eligible
        if (r.attPct < 65 || (r.attPct < 75 && !r.condoned)) {
r.grade = "SA"; r.gp = 0; r.status = "SA"; return; }
        if (r.absent) { r.grade = "AB"; r.gp = 0; r.status = "AB";
return; }
        if (r.cia1 == null || r.cia2 == null || r.asg == null ||
r.ese == null) { r.pending = true; return; }

        double ciaW, eseW, eseMinPct;
        switch (r.type) {
            case "PRACTICAL": ciaW = 0.60; eseW = 0.40; eseMinPct
= 50; break;
            case "EMBEDDED":  ciaW = 0.50; eseW = 0.50; eseMinPct
= 45; break;
            default:          ciaW = 0.40; eseW = 0.60; eseMinPct
= 45; break; // THEORY
        }
        r.scaledCia = (r.cia1 + r.cia2 + r.asg) / 3.0 * ciaW;
        r.scaledEse = r.ese * eseW;
        r.total = r.scaledCia + r.scaledEse;

        boolean pass = r.total >= 50 && r.ese >= eseMinPct;
        long t = Math.round(r.total);
        if (!pass)        { r.grade = "RA"; r.gp = 0; r.status =
"FAIL"; }
        else if (t >= 91) { r.grade = "O";  r.gp = 10; r.status =
"PASS"; }
        else if (t >= 81) { r.grade = "A+"; r.gp = 9;  r.status =
"PASS"; }
        else if (t >= 71) { r.grade = "A";  r.gp = 8;  r.status =
"PASS"; }
        else if (t >= 61) { r.grade = "B+"; r.gp = 7;  r.status =
"PASS"; }
        else              { r.grade = "B";  r.gp = 6;  r.status =
"PASS"; }
    }

    /** SGPA = sum(credits * GP) / sum(all enrolled credits). */
    public static double sgpa(List<Result> list) {
```

```
        int cr = 0, pts = 0;
        for (Result r : list) { cr += r.credits; pts += r.credits
* r.gp; }
        return cr == 0 ? 0 : (double) pts / cr;
    }

    // ---------------- Loading ----------------
    /** roll == null -> all students of the semester. */
    public List<Result> getResults(String roll, int sem) {
        String sql = "SELECT
s.roll_no,s.name,s.department,c.course_code,c.title,c.credits,c.co
urse_type,"
                +
"e.total_classes,e.attended_classes,e.is_condoned,m.cia1_marks,m.c
ia2_marks,m.assignment_marks,"
                + "m.ese_marks,m.is_ese_absent FROM enrollments e
"
                + "JOIN students s ON s.roll_no=e.roll_no JOIN
courses c ON c.course_code=e.course_code "
                + "LEFT JOIN assessment_marks m ON
m.enrollment_id=e.enrollment_id WHERE e.semester=? "
                + (roll != null ? "AND e.roll_no=? " : "") +
"ORDER BY s.roll_no,c.course_code";
        List<Result> out = new ArrayList<>();
        try (Connection c = DBConnection.getConnection();
PreparedStatement p = c.prepareStatement(sql)) {
            p.setInt(1, sem);
            if (roll != null) p.setString(2, roll);
            try (ResultSet rs = p.executeQuery()) {
                while (rs.next()) {
                    Result r = new Result();
                    r.roll = rs.getString(1); r.name =
rs.getString(2); r.dept = rs.getString(3);
                    r.code = rs.getString(4); r.title =
rs.getString(5); r.credits = rs.getInt(6); r.type =
rs.getString(7);
                    int total = rs.getInt(8), att = rs.getInt(9);
                    r.attPct = total == 0 ? 0 : att * 100.0 /
total;
                    r.condoned = rs.getBoolean(10);
                    r.cia1 = num(rs, 11); r.cia2 = num(rs, 12);
r.asg = num(rs, 13); r.ese = num(rs, 14);
                    r.absent = rs.getBoolean(15);
                    evaluate(r);
                    out.add(r);
                }
            }
        } catch (SQLException e) { System.out.println("[ERROR] " +
e.getMessage()); }
        return out;
    }

    private static Double num(ResultSet rs, int i) throws
SQLException {
        double v = rs.getDouble(i);
        return rs.wasNull() ? null : v;
    }
}
```

MAINAPP.JAVA:

```
import java.time.LocalDate;
import java.util.*;

/** Menu-driven console UI - KEC (Autonomous) Regulation 2024
Student Evaluation System. */
public class MainApp {
    private static final AcademicDAO dao = new AcademicDAO();
    private static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        if (!DBConnection.testConnection()) {
            System.out.println("Database connection failed. Check
MySQL, kec_eval DB and DBConnection settings.");
            return;
        }
        boolean running = true;
        while (running) {
            System.out.println("\n========== KEC STUDENT
EVALUATION SYSTEM - R2024 ==========");
            System.out.println(" 1. Student Registration");
            System.out.println(" 2. Course Catalog");
            System.out.println(" 3. Course Enrollment &
Attendance");
            System.out.println(" 4. Incremental Marks Entry
(CIA1/2 | Assignment | ESE)");
            System.out.println(" 5. Student Semester Mark Sheet
(SGPA)");
            System.out.println(" 6. Attendance Defaulter Report");
            System.out.println(" 7. Class Performance & Result
Summary");
            System.out.println(" 8. Medical Condonation
Approval");
            System.out.println(" 0. Exit");
            switch (ask("Enter your choice: ")) {
                case "1": registerStudent(); break;
                case "2": addCourse(); break;
                case "3": enrollStudent(); break;
                case "4": enterMarks(); break;
                case "5": markSheet(); break;
                case "6": defaulters(askInt("Semester: ")); break;
                case "7": departmentSummary(askInt("Semester: "));
break;
                case "8": condonation(); break;
                case "0": running = false; break;
                default: System.out.println("Invalid choice.");
            }
        }
        System.out.println("System closed.");
    }

    // ---------- input helpers ----------
    private static String ask(String msg) { System.out.print(msg);
return sc.nextLine().trim(); }

    private static int askInt(String msg) {
        while (true) {
            try { return Integer.parseInt(ask(msg)); }
```

```
            catch (NumberFormatException e) {
System.out.println("Enter a valid whole number."); }
        }
    }

    private static double askDouble(String msg) {
        while (true) {
            try { return Double.parseDouble(ask(msg)); }
            catch (NumberFormatException e) {
System.out.println("Enter a valid number."); }
        }
    }

    // ---------- data entry ----------
    private static void registerStudent() {
        String roll = ask("Roll Number (e.g. 24ITR001):
").toUpperCase();
        String name = ask("Student Name: ");
        String dept = ask("Department (CSE/ECE/MECH/IT...): ");
        int sem = askInt("Current Semester (1-8): ");
        System.out.println(dao.addStudent(roll, name, dept, sem) ?
"[SUCCESS] Student registered." : "[FAILED] Could not register.");
    }

    private static void addCourse() {
        String code = ask("Course Code (e.g. 24ITR31):
").toUpperCase();
        String title = ask("Course Title: ");
        int credits = askInt("Credits: ");
        String type = ask("Course Type (1.THEORY 2.PRACTICAL
3.EMBEDDED): ");
        type = type.equals("1") ? "THEORY" : type.equals("2") ?
"PRACTICAL" : type.equals("3") ? "EMBEDDED" : "";
        if (type.isEmpty()) { System.out.println("Invalid course
type."); return; }
        int sem = askInt("Offering Semester (1-8): ");
        System.out.println(dao.addCourse(code, title, credits,
type, sem)
                ? "[SUCCESS] Course " + code + " [" + type + ", "
+ credits + " Credits] added." : "[FAILED] Could not add
course.");
    }

    private static void enrollStudent() {
        String roll = ask("Roll Number: ").toUpperCase();
        String course = ask("Course Code: ").toUpperCase();
        int sem = askInt("Semester: ");
        int total = askInt("Total Classes Held: ");
        int attended = askInt("Classes Attended: ");
        if (attended > total || attended < 0) {
System.out.println("Attended classes cannot exceed total
classes."); return; }
        double pct = total == 0 ? 0 : attended * 100.0 / total;
        System.out.printf("Attendance: %.2f%%%n", pct);
        if (pct >= 75) System.out.println("[INFO] Eligible for
ESE.");
        else if (pct >= 65) System.out.println("[INFO]
Conditionally eligible - needs Medical Condonation (menu 8).");
        else System.out.println("[INFO] Shortage of Attendance -
Grade SA, barred from ESE.");
```

```
        int id = dao.enroll(roll, course, sem, total, attended);
        System.out.println(id > 0 ? "[SUCCESS] Enrollment saved.
(ID: " + id + ")" : "[FAILED] Check roll number / course code.");
    }

    private static void condonation() {
        int id = dao.getEnrollmentId(ask("Roll Number:
").toUpperCase(), ask("Course Code: ").toUpperCase(),
askInt("Semester: "));
        if (id <= 0) { System.out.println("Enrollment not
found."); return; }
        boolean ok = ask("Medical condonation approved? (y/n):
").equalsIgnoreCase("y");
        System.out.println(dao.setCondonation(id, ok) ? "[SUCCESS]
Updated." : "[FAILED]");
    }

    private static void enterMarks() {
        String roll = ask("Roll Number: ").toUpperCase();
        String course = ask("Course Code: ").toUpperCase();
        int sem = askInt("Semester: ");
        int id = dao.getEnrollmentId(roll, course, sem);
        if (id <= 0) { System.out.println("Enrollment not
found."); return; }
        System.out.println("1. Phase 1 (Mid-Sem): CIA Test 1 &
2");
        System.out.println("2. Phase 2 (End of Sem CIA):
Assignment/Quiz/Lab");
        System.out.println("3. Phase 3 (Post Sem): ESE marks /
Absent (AB)");
        boolean ok;
        switch (ask("Phase (1-3): ")) {
            case "1": ok = dao.recordPhase1Marks(id,
askDouble("CIA Test 1 (0-100): "), askDouble("CIA Test 2 (0-100):
")); break;
            case "2": ok = dao.recordPhase2Marks(id,
askDouble("Assignment/Quiz/Lab (0-100): ")); break;
            case "3":
                boolean ab = ask("Absent in ESE? (y/n):
").equalsIgnoreCase("y");
                ok = dao.recordPhase3Marks(id, ab ? null :
askDouble("ESE marks (0-100): "), ab); break;
            default: System.out.println("Invalid phase."); return;
        }
        System.out.println(ok ? "[SUCCESS] Marks recorded." :
"[FAILED] Marks not saved.");
    }

    // ---------- reports ----------
    private static void markSheet() {
        String roll = ask("Roll Number: ").toUpperCase();
        int sem = askInt("Semester: ");
        List<AcademicDAO.Result> list = dao.getResults(roll, sem);
        if (list.isEmpty()) { System.out.println("No enrollments
found."); return; }
        String line = "=".repeat(128);
        System.out.println("\n" + line);
        System.out.println(center("KONGU ENGINEERING COLLEGE
(AUTONOMOUS)", 128));
```

```
        System.out.println(center("GRADE SHEET - REGULATION 2024",
128));
        System.out.println(line);
        System.out.printf("Roll No: %s   Name: %s   Dept: %s
Semester: %d   Date: %s%n",
                roll, list.get(0).name, list.get(0).dept, sem,
LocalDate.now());
        System.out.println("-".repeat(128));
        System.out.printf("%-9s %-28s %-9s %2s %6s %6s %6s %6s %7s
%7s %7s %7s %-4s %-8s%n",
                "Course", "Title", "Type", "Cr", "Att%", "CIA1",
"CIA2", "Asg", "Sc.CIA", "ESE", "Sc.ESE", "Total", "Gr",
"Result");
        System.out.println("-".repeat(128));
        int earned = 0, enrolled = 0; int pts = 0;
        boolean arrears = false, pending = false;
        for (AcademicDAO.Result r : list) {
            enrolled += r.credits; pts += r.credits * r.gp;
            if (r.gp > 0) earned += r.credits;
            if (r.pending) pending = true; else if (r.gp == 0)
arrears = true;
            System.out.printf("%-9s %-28s %-9s %2d %6.1f %6s %6s
%6s %7s %7s %7s %7s %-4s %-8s%n",
                    r.code, cut(r.title, 28), r.type, r.credits,
r.attPct, f(r.cia1), f(r.cia2), f(r.asg),
                    r.pending || r.gp == 0 && r.total == 0 ? "-" :
String.format("%.2f", r.scaledCia),
                    r.absent ? "AB" : f(r.ese),
                    r.pending || r.gp == 0 && r.total == 0 ? "-" :
String.format("%.2f", r.scaledEse),
                    r.pending || r.gp == 0 && r.total == 0 ? "-" :
String.format("%.2f", r.total),
                    r.grade, r.status);
        }
        double sgpa = AcademicDAO.sgpa(list);
        System.out.println("-".repeat(128));
        System.out.printf("Total Credits: %d | Credits Earned: %d
| Weighted Grade Points: %d%n", enrolled, earned, pts);
        System.out.printf("SGPA: %.2f / 10.00%s%n", sgpa, pending
? "  (provisional - some marks pending)" : "");
        System.out.printf("Equivalent Percentage: %.2f%%%n", sgpa
* 10);
        System.out.println("Outcome: " + (pending ? "INCOMPLETE" :
arrears ? "ARREARS - RE-APPEARANCE REQUIRED" : "PASSED - ALL
COURSES CLEARED"));
        System.out.println(line);
        System.out.println("Grades: O(91-100)=10 | A+(81-90)=9 |
A(71-80)=8 | B+(61-70)=7 | B(50-60)=6 | RA/SA/AB=0");
    }

    private static void defaulters(int sem) {
        List<AcademicDAO.Result> list = dao.getResults(null, sem);
        System.out.println("\nATTENDANCE DEFAULTER REPORT -
SEMESTER " + sem);
        System.out.printf("%-10s %-18s %-6s %-9s %-7s %-14s%n",
"Roll No", "Name", "Dept", "Course", "Att%", "Status");
        System.out.println("-".repeat(75));
        int n = 0;
        for (AcademicDAO.Result r : list) {
            if (r.attPct >= 75) continue;
```

```
            String st = r.attPct < 65 ? "SA (Barred)" : r.condoned
? "Condoned" : "SA (No condonation)";
            System.out.printf("%-10s %-18s %-6s %-9s %6.1f%%
%-14s%n", r.roll, cut(r.name, 18), r.dept, r.code, r.attPct, st);
            n++;
        }
        if (n == 0) System.out.println("No defaulters found.");
        System.out.println("Note: <65% -> SA (barred) | 65-74.9%
-> needs Medical Condonation");
    }

    private static void departmentSummary(int sem) {
        Map<String, List<AcademicDAO.Result>> byStudent = new
LinkedHashMap<>();
        for (AcademicDAO.Result r : dao.getResults(null, sem))
byStudent.computeIfAbsent(r.roll, k -> new ArrayList<>()).add(r);

        class Stat { int total, passed, failed, incomplete; double
sgpaSum; int counted; }
        Map<String, Stat> depts = new TreeMap<>();
        for (List<AcademicDAO.Result> courses :
byStudent.values()) {
            Stat s = depts.computeIfAbsent(courses.get(0).dept, k
-> new Stat());
            s.total++;
            boolean pend = courses.stream().anyMatch(r ->
r.pending);
            boolean fail = courses.stream().anyMatch(r ->
!r.pending && r.gp == 0);
            if (pend) { s.incomplete++; continue; }
            if (fail) s.failed++; else s.passed++;
            s.sgpaSum += AcademicDAO.sgpa(courses); s.counted++;
        }
        System.out.println("\nDEPARTMENT-WISE RESULT SUMMARY -
SEMESTER " + sem);
        System.out.printf("%-10s %8s %8s %12s %11s %10s %9s%n",
"Dept", "Students", "Passed", "With Arrear", "Incomplete", "Pass
%", "Avg SGPA");
        System.out.println("-".repeat(75));
        for (Map.Entry<String, Stat> e : depts.entrySet()) {
            Stat s = e.getValue();
            double pr = s.counted == 0 ? 0 : s.passed * 100.0 /
s.counted;
            double avg = s.counted == 0 ? 0 : s.sgpaSum /
s.counted;
            System.out.printf("%-10s %8d %8d %12d %11d %9.2f%%
%9.2f%n", e.getKey(), s.total, s.passed, s.failed, s.incomplete,
pr, avg);
        }
    }

    private static String f(Double d) { return d == null ? "-" :
String.format("%.1f", d); }
    private static String cut(String s, int n) { return s.length()
<= n ? s : s.substring(0, n - 1) + "."; }
    private static String center(String s, int w) { return "
".repeat(Math.max(0, (w - s.length()) / 2)) + s; }
}
```

OUTPUT SCREENSHOTS:

MAIN MENU:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
==================== KEC STUDENT EVALUATION SYSTEM - R2024 ====================
 1. Student Registration
 2. Course Catalog
 3. Course Enrollment & Attendance
 4. Incremental Marks Entry (CIA1/2 | Assignment | ESE)
 5. Student Semester Mark Sheet (SGPA)
 6. Attendance Defaulter Report
 7. Class Performance & Result Summary
 8. Medical Condonation Approval
 0. Exit
--------------------------------------------------------------------------------
Enter your choice: 1
```

1. STUDENT REGISTRATION:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 1
Roll Number (e.g. 24ITR001): 25ITR152
Student Name: Swetha B
Department (CSE/ECE/MECH/IT...): IT
Current Semester (1-8): 3
[SUCCESS] Student registered.
Enter your choice: _
```

2.Course Catalog

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 2
Course Code (e.g. 24ITR31): 24ITT32
Course Title: Programming in Java
Credits: 3
Course Type (1.THEORY 2.PRACTICAL 3.EMBEDDED): 1
Offering Semester (1-8): 3
[SUCCESS] Course 24ITT32 [THEORY, 3 Credits] added.
Enter your choice: _
```

3. COURSE ENROLLMENT & ATTENDANCE TRACKING:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 3
Roll Number: 25ITR152
Course Code: 24ITT32
Semester: 3
Total Classes Held: 40
Classes Attended: 36
Attendance: 90.00%
[INFO] Eligible for ESE.
[SUCCESS] Enrollment saved. (ID: 1)
Enter your choice: _
```

4. PHASE 1 (MID-SEMESTER) - CIA 1 & CIA 2:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 4
Roll Number: 25ITR152
Course Code: 24ITT32
Semester: 3
1. Phase 1 (Mid-Sem): CIA Test 1 & 2
2. Phase 2 (End of Sem CIA): Assignment/Quiz/Lab
3. Phase 3 (Post Sem): ESE marks / Absent (AB)
Phase (1-3): 1
CIA Test 1 (0-100): 82
CIA Test 2 (0-100): 86
[SUCCESS] Marks recorded.
```

5. PHASE 2 (END OF SEMESTER) - ASSIGNMENT:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 4
Roll Number: 25ITR152
Course Code: 24ITT32
Semester: 3
Phase (1-3): 2
Assignment/Quiz/Lab (0-100): 90
[SUCCESS] Marks recorded.
Enter your choice: _
```

6. PHASE 3 (POST SEMESTER) - ESE MARKS:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 4
Roll Number: 25ITR152
Course Code: 24ITT32
Semester: 3
Phase (1-3): 3
Absent in ESE? (y/n): n
ESE marks (0-100): 84
[SUCCESS] Marks recorded.
Enter your choice: _
```

7. ATTENDANCE DEFAULTER REPORT:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 6
ATTENDANCE DEFAULTER REPORT - SEMESTER 3
Roll No    Name               Dept   Course    Att%    Status
------------------------------------------------------------------------
25ITR101  Kavin R             IT     24ITT32   62.5    SA (Barred)
25ITR108  Divya S             IT     24ITT32   70.0    SA (No condonation)
------------------------------------------------------------------------
Note: <65% -> SA (barred) | 65-74.9% -> needs Medical Condonation
```

8. STUDENT SEMESTER MARK SHEET (SGPA & PERCENTAGE):

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 5
================ KONGU ENGINEERING COLLEGE (AUTONOMOUS) ================
                   GRADE SHEET - REGULATION 2024
Roll No: 25ITR152  Name: Swetha B  Dept: IT  Semester: 3
------------------------------------------------------------------------------
Course   Title                 Type       Cr  Att% CIA1 CIA2 Asg Sc.CIA ESE
------------------------------------------------------------------------------
24ITT32  Programming in Java    THEORY      3  90.0  82   86  90  34.40 84
24ITL31  Java Laboratory        PRACTICAL   1  88.0  90   85  92  53.40 86
------------------------------------------------------------------------------
Total Credits: 4 | Credits Earned: 4 | Weighted Grade Points: 36
SGPA: 9.00 / 10.00
Equivalent Percentage: 90.00%
Outcome: PASSED - ALL COURSES CLEARED
```

9. DEPARTMENT-WISE CLASS PERFORMANCE & RESULT SUMMARY:

```
C:\KEC_Project> java -cp ".;mysql-connector-j-8.0.xx.jar" MainApp
Enter your choice: 7
DEPARTMENT-WISE RESULT SUMMARY - SEMESTER 3
Dept       Students   Passed   With Arrear   Incomplete   Pass %   Avg SGPA
--------------------------------------------------------------------------------
IT              3        2             1            0    66.67%      8.25
CSE             2        2             0            0   100.00%      8.60
ECE             1        0             1            0     0.00%      0.00
```

10.DATABASE TABLE

```
C:\KEC_Project> mysql -u root -p
mysql> SELECT roll_no,name,department,current_semester FROM students;
+----------+----------+------------+------------------+
| roll_no  | name     | department | current_semester |
+----------+----------+------------+------------------+
| 25ITR152 | Swetha B | IT         | 3                |
| 25ITR101 | Kavin R  | IT         | 3                |
| 25ITR108 | Divya S  | IT         | 3                |
+----------+----------+------------+------------------+
3 rows in set
mysql> SELECT * FROM courses;
+------------+----------------------+---------+-------------+----------+
| 24ITT32    | Programming in Java  | 3       | THEORY      | 3        |
+------------+----------------------+---------+-------------+----------+
1 row in set
```

RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) Regulation 2024 was successfully designed, developed, and tested using Java (JDBC) and MySQL. The system automates the calculation of Continuous Internal Assessments (CIA), End Semester Examinations (ESE), letter grades, attendance eligibility, and Semester Grade Point Average (SGPA).

REFERENCES:

1. Oracle, Java SE Documentation - JDBC Overview and Architecture. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

2. Oracle, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

3. Oracle, MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connectorj/8.0/en/

4. University Grants Commission, Guidelines for Choice Based Credit System (CBCS). https://www.ugc.gov.in/

5. GeeksforGeeks, Establishing JDBC Connection in Java. https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/

6. Anthropic, Claude AI Assistant (used for project guidance). https://claude.ai/

GITHUB:

LINK: https://github.com/sushmi0307/java-